CODEMITVISHAL · FREE COURSE

Python Pro

From Python Foundations to **professional** Python: clean code, data structures, OOP, testing, packaging, concurrency and real projects.

13 modules

60+ runnable examples

0 € always free

github.com/CodeMitVishal/python-pro

COURSE MAP

From foundations to pro, in 13 modules

#	MODULE	YOU'LL BE ABLE TO…
08	**Pythonic code**	write short, idiomatic Python
09	**Data structures deep dive**	pick the right structure (Big-O)
10	**Advanced functions**	write closures and decorators
11	**Iterators & generators**	process data lazily
12	**Advanced OOP**	design clean classes (SOLID)
13	**Errors, context managers, logging**	fail safely, log clearly
14	**Project structure & packaging**	ship an installable CLI

#	MODULE	YOU'LL BE ABLE TO…
15	**Type hints & code quality**	catch bugs before running
16	**Testing with pytest**	prove your code works
17	**Concurrency**	use threads, processes, asyncio
18	**Data, APIs & databases**	build a REST API with a DB
19	**Performance & internals**	profile and speed things up
20	**Capstone project**	build it all, end to end

Every module: slides · runnable code · exercise + solution · quiz

github.com/CodeMitVishal/python-pro

MODULE

08

Pythonic code

Same results, half the code. Write Python the way experienced Python developers do.

IN THIS MODULE

Comprehensions	01_comprehensions.py
Unpacking	02_unpacking.py
Looping like a pro	03_looping_like_a_pro.py
Truthiness & the walrus :=	04_truthiness_and_walrus.py
Strings like a pro	05_strings_like_a_pro.py
Demo: grade report	06_demo_grade_report.py
Exercise: refactor to Pythonic	ex08_refactor_to_pythonic.py

LESSON · COMPREHENSIONS

Build a list in one readable line

```
[ p * 1.19 for p in prices if p < 20 ]
```

① what to keep ② the loop ③ optional filter

```
# the old way: 4 lines
with_tax = []
for p in prices:
    with_tax.append(p * 1.19)

# Pythonic: 1 line
with_tax = [p * 1.19 for p in prices]
```

KIND	SYNTAX
list	`[x for x in xs]`
dict	`{k: v for k, v in pairs}`
set	`{x.lower() for x in xs}`
generator	`sum(x for x in xs)`

> **if/else before for** chooses a value. **if after for** filters.

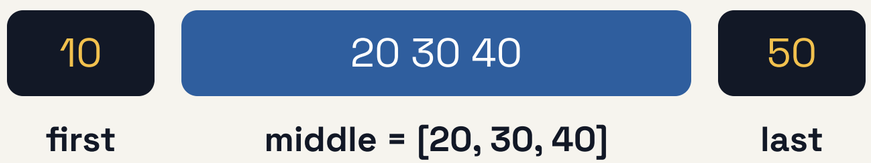
LESSON · UNPACKING

Pull values out and into names, in one step

```
x, y = (3, 7)                  # basic
a, b = b, a                    # swap, no temp
first, *middle, last = nums    # * = "the rest"
name, _, city = record         # _ = ignore

area(*[4, 5])                  # list -> args
area(**{"width": 2, "height": 9})

merged = [*odds, *evens]
settings = defaults | user     # later keys win
```



> The number of names must match the number of values, **unless** one name has a `*`.

> `*` spreads a list into arguments, `**` spreads a dict into keyword arguments.

LESSON · LOOPING LIKE A PRO

Stop writing range(len(...))

INSTEAD OF…	WRITE
`for i in range(len(xs)): xs[i]`	`for i, x in enumerate(xs, start=1)`
Two lists with one index	`for a, b in zip(xs, ys)`
A custom sort function	`sorted(xs, key=len, reverse=True)`
A flag + loop + break	`any(x < 0 for x in xs)`
”Is every item OK?” loop	`all(x >= 50 for x in xs)`
Track the biggest by hand	`max(pairs, key=lambda p: p[1])`
`for k in d: d[k]`	`for k, v in d.items()`

```
for s in scores:
    if s < 50:
        print("fail found")
        break
else:
    print("all passed")
```

> **for … else:** the `else` runs only if the loop finished **without** `break`.

> `zip` stops at the shortest list. Add `strict=True` to catch length bugs.

LESSON · TRUTHINESS & WALRUS

Short, clear conditions

FALSY

`0 0.0 "" [] {} set() None False`

Everything else is truthy, even `"0"` and `[0]`.

INSTEAD OF…	WRITE
`if len(cart) == 0:`	`if not cart:`
`if x == None:`	`if x is None:`
`x >= 50 and x < 80`	`50 <= x < 80`
if/else to pick a default	`name or "Guest"`

```
# walrus := assigns AND returns
while (line := input()) != "quit":
    print(line.upper())

if (n := len(data)) > 3:
    print(f"long list: {n} items")
```

⚠️ `count or 10` turns a real **0** into 10, because 0 is falsy. Use `10 if count is None else count`.

LESSON · STRINGS LIKE A PRO

f-strings format, align and even debug

SPEC	EXAMPLE	RESULT
`.2f`	`f"{1234.5678:.2f}"`	**1234.57**
`,.2f`	`f"{1234.5:,.2f}"`	**1,234.50**
`.1%`	`f"{0.4567:.1%}"`	**45.7%**
`05d`	`f"{42:05d}"`	**00042**
`<10 / >10`	`f"{name:<10}|"`	**left / right align**
`=`	`f"{user=}"`	**user='Vishal'**

```
" ".join(["learn", "python"])
# 'learn python'

name, age, city = line.split(",")
# split + unpack in one go
```

> Strings are **immutable**: every method returns a **new** string.

> Build big text with a list and `"".join()`, not `+=` in a loop.

DEMO · GRADE REPORT

Everything from Module 8 in one program

```
for line in raw.splitlines():
    name, *marks = line.split(",")          # unpacking
    students[name] = [int(m) for m in marks]  # list comp

averages = {n: sum(m) / len(m)                 # dict comp
            for n, m in students.items()}
ranking = sorted(averages.items(),
                 key=lambda p: p[1], reverse=True)

for place, (name, avg) in enumerate(ranking, 1):
    result = "PASS" if avg >= 50 else "FAIL"
    print(f"{place:<3}{name:<10}{avg:>6.1f}  {result}")

print(all(a >= 50 for a in averages.values()))
```

OUTPUT

```
#  Name          Avg  Result
1  Vishal       91.3  PASS
2  Li           85.3  PASS
3  Raj          70.7  PASS
4  Sam          45.0  FAIL

Everyone passed?  False
```

> Run it: `python 08_pythonic_code/06_demo_grade_report.py`

EXERCISE 8 · YOUR TURN

Refactor 9 functions to Pythonic code

The task

> Each function works, but it's written the long way. Rewrite each one with the **Module 8 tool that fits**. Keep the names and results the same.

> `money()` also has a bug: `1234.5` must become **€1,234.50**.

> Run the file: all built-in checks must pass.

comprehension · enumerate · zip · any · max(key=) · or · join

```
# before
result = []
for n in numbers:
    if n % 2 == 0:
        result.append(n * n)

# after
[n * n for n in numbers if n % 2 == 0]
```

Starter:
exercises/ex08_refactor_to_pythonic.py
Solution:
solutions/ex08_refactor_to_pythonic.py

MODULE

09

Data structures deep dive

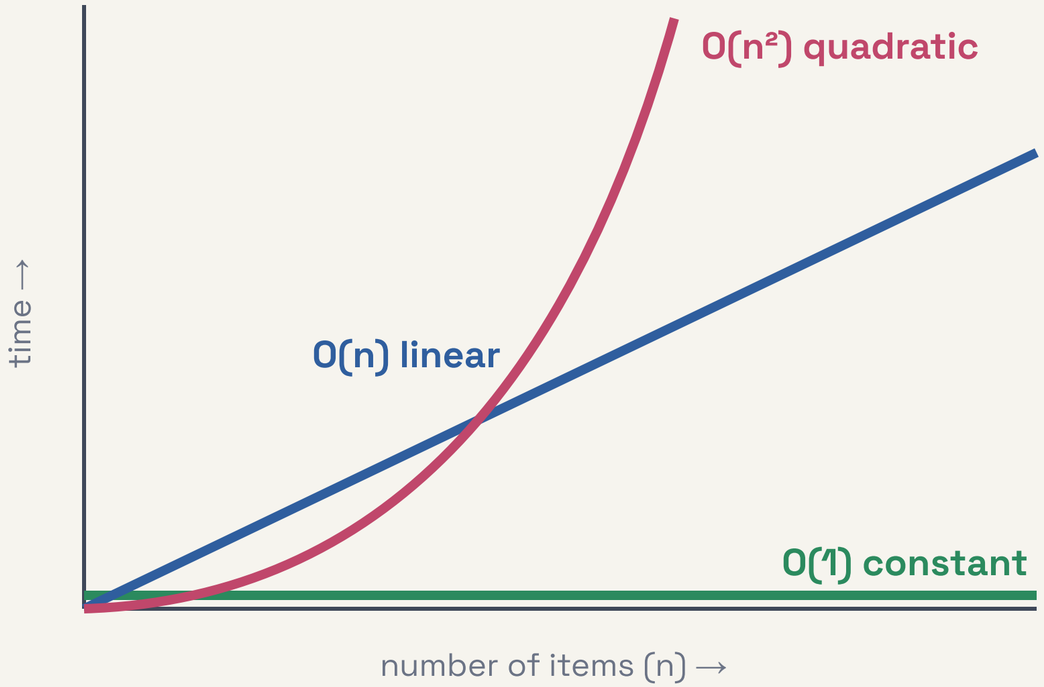
How lists, dicts and sets work inside, and how to pick the right one so your code stays fast.

IN THIS MODULE

Big-O: why it matters	01_big_o.py
Lists & tuples under the hood	02_lists_and_tuples.py
Dicts, sets & hashing	03_dicts_and_sets.py
The collections module	04_collections.py
dataclasses	05_dataclasses.py
Demo: log analyzer	06_demo_log_analyzer.py
Exercise: library tracker	ex09_library_tracker.py

LESSON · BIG-O

How does the work grow when the data grows?



OPERATION	LIST	DICT / SET
`x in …`	O(n)	O(1)
read by index / key	O(1)	O(1)
add at the end	O(1)	O(1)
insert at the front	O(n)	–

MEASURED: "is the last item in here?" × 200

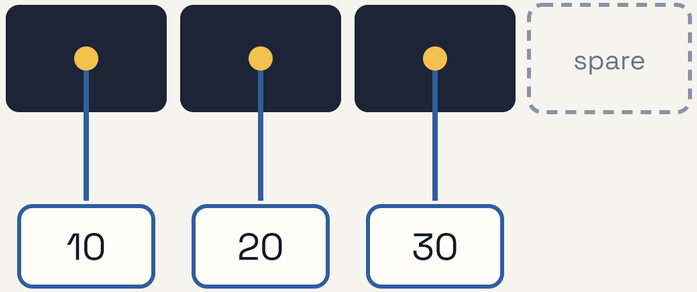
```
    1,000 items | list:    1 ms | set: 0.01 ms
  100,000 items | list:  176 ms | set: 0.02 ms
1,000,000 items | list: 2304 ms | set: 0.01 ms
```

Checking membership a lot? Convert to a **set** once.

LESSON · LISTS & TUPLES UNDER THE HOOD

A list is a row of references, with spare room

a = [10, 20, 30] (capacity 4)



append → spare slot: **fast**

insert(0, x) → shift all: **slow**

each slot = a reference

```
b = a           # same list, 2 names
c = a.copy()    # new outer list (shallow)
d = copy.deepcopy(grid)  # copies inner lists too
```

LIST

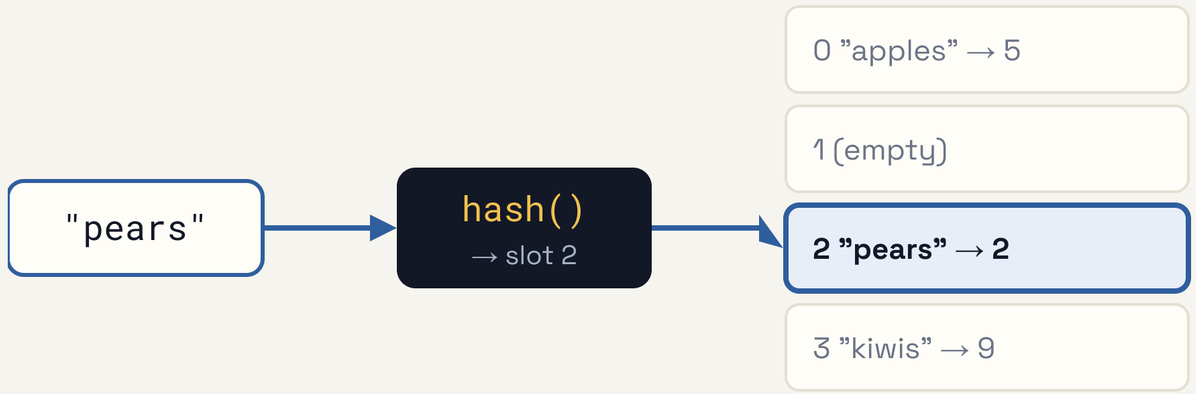
Changeable. Grows and shrinks. Use for **collections of similar items**.

TUPLE

Read-only and lighter (56 vs 72 bytes for 2 items). **Hashable**, so it can be a dict key. Use for **fixed records**: `(x, y)`, RGB, DB rows.

⚠️ A shallow copy shares the inner lists. Nested data? Use **copy.deepcopy**.

Dicts don’t search. They jump straight to the answer.



One calculation, no matter how many keys → **O(1)**

> Keys must be **hashable** = immutable: `str` `int` `tuple` `frozenset`. Not list, dict or set.

SET OP	MEANS
`a | b`	in either (union)
`a & b`	in both (intersection)
`a - b`	only in a (difference)
`a ^ b`	in exactly one

> Dicts keep **insertion order**. Dedupe and keep order: `list(dict.fromkeys(xs))`

LESSON · COLLECTIONS

Four containers that delete boilerplate

COUNTER

Count anything

```
Counter(words).most_common(2)
# [('the', 3), ('and', 2)]
```

DEFAULTDICT

No more "if key not in dict"

```
by_team = defaultdict(list)
by_team[team].append(name)
```

DEQUE

Fast at both ends · keep the last N

```
recent = deque(maxlen=3)
recent.append(page)  # oldest drops off
```

NAMEDTUPLE

A tuple with field names

```
Point = namedtuple("Point", ["x", "y"])
Point(3, 7).x   # 3
```

LESSON · DATACLASSES

Classes for data, without the boilerplate

```python
@dataclass
class Product:
    name: str
    price: float
    qty: int = 0                                # default
    tags: list = field(default_factory=list)

    def total(self) -> float:
        return self.price * self.qty

Product("Laptop", 899.0, 2)
# Product(name='Laptop', price=899.0, qty=2, tags=[])
```

YOU GET FOR FREE

__init__ · **__repr__** · **__eq__**, built from the fields.

OPTIONS

`frozen=True` → read-only and hashable
`order=True` → sortable by the fields
`asdict(obj)` → plain dict (great for JSON)

⚠️ Never `tags: list = []`. Every object would share one list. Use **field(default_factory=list)**.

DEMO · LOG ANALYZER

One question, one right data structure

QUESTION	TOOL
One clean record per log line?	**@dataclass Entry**
Most popular pages?	**Counter** → `most_common(3)`
Which pages did each user visit?	**defaultdict(set)**
How many unique visitors?	**set** comprehension
The last 3 server errors?	**deque(maxlen=3)**

OUTPUT

```
Top pages:
  /home  4 hits
  /cart  3 hits
  /shop  2 hits

Unique visitors: 4

Last 3 server errors:
  10:04 /cart (raj)
  10:07 /cart (vishal)
  10:09 /cart (li)
```

EXERCISE 9 · YOUR TURN

Build a library tracker

The task: fill in 6 TODOs

1	Make `Book` a **dataclass** (isbn, title, author)
2	`build_index` : a **dict** isbn → Book (O(1) lookups)
3	`most_borrowed` : top n with **Counter**
4	`titles_by_author` : **defaultdict(list)**
5	`unique_borrowers` : a **set**
6	`ReturnLog` : the last 3 returns with **deque(maxlen=3)**

```
$ python exercises/
    ex09_library_tracker.py
All checks passed!
```

Starter:
exercises/ex09_library_tracker.py
Solution:
solutions/ex09_library_tracker.py

> Done? Open a **Pull Request** and I'll review your code.

MODULE

10

Advanced functions

Functions are objects: pass them, return them, make them remember, wrap them.

IN THIS MODULE

Parameters like a pro	01_args_and_kwargs.py
Scope & closures	02_scope_and_closures.py
Functions are objects	03_first_class_functions.py
Decorators	04_decorators.py
functools	05_functools.py
Demo: resilient API client	06_demo_api_client.py
Exercise: write decorators	ex10_decorators.py

LESSON · PARAMETERS

*args, **kwargs, keyword-only and the trap

```
def f(a, b=2, *args, c, d=4, **kwargs):
#     |  |     |      |  |     └ extra keywords -> dict
#     |  |     |      └──┘ keyword-only (after *)
#     |  |     └ extra positionals -> tuple
#     |  └ default
#     └ positional

def connect(host, *, port=443): ...
connect("api.io", port=8080)    # must name it

def add(item, basket=None):     # NOT basket=[]
    if basket is None:
        basket = []
    basket.append(item)
    return basket
```

> ***args** collects extra positional arguments into a **tuple**, ****kwargs** extra keywords into a **dict**.

> Everything after a bare `*` must be passed **by name**: call sites explain themselves.

⚠️ Default values are created **once**. A `[]` default is shared by every call. Use **None**.

LESSON · SCOPE & CLOSURES

A function that remembers

```
def make_counter():
    count = 0
    def next_number():
        nonlocal count      # update the OUTER count
        count += 1
        return count
    return next_number      # the function, not a call

c1, c2 = make_counter(), make_counter()
c1(), c1(), c1(), c2()      # 1 2 3 1
```

LEGB: Python looks for a name in **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in.

A **closure** keeps the variables of its outer function alive, even after it returned.

Reading an outer variable just works. **Assigning** needs `nonlocal` (or `global`).

⚠️ `[lambda: i for i in range(3)]` all return 2 (late binding). Fix: **lambda i=i: i**

LESSON · DECORATORS

Wrap a function to add behaviour

```
def timer(func):
    @functools.wraps(func)          # keep name + docstring
    def wrapper(*args, **kwargs):   # accept ANY arguments
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__}: {time.perf_counter() - start:.3f}s")
        return result               # don't forget!
    return wrapper

@timer                              # same as: slow = timer(slow)
def slow(): ...
```

> **@decorator** above `def f` means `f = decorator(f)`.

> Decorator **with arguments** = one more layer: `retry(times)` → `decorator(func)` → `wrapper(*args)`.

> Stacked decorators: the one **closest to def** wraps first.

LESSON · FUNCTOOLS

Ready-made tools for functions

TOOL	WHAT IT DOES	EXAMPLE
lru_cache	remembers results (memoization)	`fib(200)` : instant
partial	pre-fills arguments	`square = partial(power, exp=2)`
wraps	keeps names in decorators	`@functools.wraps(func)`
reduce	folds a list into one value	`reduce(mul, [1,2,3,4])` → 24
singledispatch	one function, code per type	`describe(21)` vs `describe([1])`

```
@lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)

fib(28)   # 0.064 s -> 0.00003 s
```

Only cache **pure** functions: same input → same output, no side effects.

EXERCISE 10 · YOUR TURN

Write three decorators

The task

1	`@count_calls` : store the number of calls in `func.calls`
2	`@require_positive` : **ValueError** for negative numbers
3	`@memoize` : your own cache with a dict (no lru_cache!)
4	Work with any arguments and keep the name: **functools.wraps**

```
@count_calls
def hello(): return "hi"

hello(); hello(); hello()
hello.calls        # 3
```

Starter: exercises/ex10_decorators.py
Solution: solutions/ex10_decorators.py

> Done? Open a **Pull Request** and I'll review your code.

MODULE

11

Iterators & generators

Process data one item at a time: a 10 GB file with the memory of 10 lines.

IN THIS MODULE

How for really works	`01_iteration_protocol.py`
Generators	`02_generators.py`
Lazy pipelines	`03_pipelines.py`
itertools	`04_itertools.py`
Demo: a 200,000-row CSV	`05_demo_big_csv.py`
Exercise: lazy generators	`ex11_generators.py`

LESSON · THE ITERATION PROTOCOL

What a for loop really does

```
for x in things:           # is really:
    print(x)

it = iter(things)          # iterable -> iterator
while True:
    try:
        x = next(it)       # next item...
    except StopIteration: # ...until there's none
        break
    print(x)
```

Iterable: something you can loop over. It hands out an iterator (`__iter__`).

Iterator: remembers **where you are** and gives the next item (`__next__`).

An iterator is **used up** after one pass: `list(it)` twice gives `[]` the 2nd time.

LESSON · GENERATORS

yield: hand out a value, pause, resume

```
def countdown(start):
    while start > 0:
        yield start        # hand out + PAUSE here
        start -= 1         # resume here on next()

list(countdown(3))         # [3, 2, 1]

squares = (n*n for n in range(10**6))  # generator expression
sum(squares)               # one value at a time
```

> Calling a generator function runs **nothing**; it returns a generator object.

> Each `next()` runs to the next `yield` and keeps all local variables.

> `yield from xs` hands over to another iterable.

Memory: a list of 1M squares = **8.4 MB**, the generator = **200 bytes**.

LESSON · LAZY PIPELINES

Chain generators like an assembly line

```
lines   = read_lines(path)          # yields one line at a time
records = parse(lines)              # yields (date, level, msg)
errors  = only("ERROR", records)    # filters lazily
msgs    = messages(errors)          # formats lazily

for m in islice(msgs, 2):           # take 2, then stop
    print(m)
# only 4 lines were ever read!
```

> Nothing runs until something **asks** for values.

> Each stage does **one job**, so it's easy to test and reuse.

> Demo with a 200,000-row CSV: **56 MB** (list) vs **0.05 MB** (generators).

LESSON · ITERTOOLS

A toolbox of fast, lazy iterators

TOOL	EXAMPLE	RESULT
islice	`islice(count(10, 5), 4)`	10 15 20 25
chain	`chain([1, 2], "ab")`	1 2 'a' 'b'
accumulate	`accumulate([5, 10, 20])`	5 15 35
pairwise	`pairwise([12, 15, 11])`	(12,15) (15,11)
groupby	sort first, then group neighbours	DE 150 · IN 100
product	`product("SM", ["red", "blue"])`	every combination
combinations	`combinations(names, 2)`	every pair

⚠️ **groupby** only groups **consecutive** items. Sort by the same key first.

EXERCISE 11 · YOUR TURN

Write lazy generators

The task

1	`chunked(items, size)` : lists of `size` items
2	`running_average(numbers)` : the average so far
3	`valid_numbers(lines)` : ints only, skip the rest
4	Use **yield**; must work on **endless** input

```
list(chunked("abcde", 2))
# [['a','b'], ['c','d'], ['e']]

list(running_average([2, 4, 6]))
# [2.0, 3.0, 4.0]
```

Starter: exercises/ex11_generators.py
Solution: solutions/ex11_generators.py

> Done? Open a **Pull Request** and I'll review your code.

MODULE

12

Advanced OOP

Classes that feel built-in, and designs that stay easy to change.

IN THIS MODULE

Dunder methods	`01_dunder_methods.py`
Properties	`02_properties.py`
Class & static methods	`03_class_and_static_methods.py`
Inheritance & the MRO	`04_inheritance_and_mro.py`
ABCs & Protocols	`05_abc_and_protocols.py`
Composition & SOLID	`06_composition_and_solid.py`
Exercise: a Vector class	`ex12_vector.py`

LESSON · DUNDER METHODS

Python calls them for you

YOU WRITE	PYTHON CALLS
`print(x)` / `repr(x)`	`__str__` / `__repr__`
`a == b`, `a < b`	`__eq__`, `__lt__` (+ @total_ordering)
`a + b`, `2 * v`	`__add__`, `__rmul__`
`len(x)`, `x[i]`, `for i in x`	`__len__`, `__getitem__`
`item in x`	`__contains__`
`{x}`, dict key	`__hash__`
`if x:`	`__bool__`

```
Money(10.5) + Money(4.25)
# 14.75 EUR
sorted([Money(9), Money(1)])
```

> Can't compare with that type? Return **NotImplemented**, not False.

Defined **__eq__**? Define **__hash__** too, or the objects can't go in sets.

Attributes with rules, alternative constructors

```python
class Temperature:
    @property
    def celsius(self):
        return self._celsius
    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("below absolute zero!")
        self._celsius = value

class Employee:
    @classmethod
    def from_csv(cls, line):       # cls = the class it's called on
        name, salary = line.split(",")
        return cls(name, int(salary))
```

@property: looks like an attribute, runs a method: validation, computed or read-only values.

@classmethod: alternative constructors. `Manager.from_csv()` returns a **Manager**.

@staticmethod: a plain helper that lives in the class.

One clear order, explicit contracts

```
class Child(Left, Right): ...
Child.__mro__  # Child, Left, Right, Base, object

class Shape(ABC):                  # nominal: must inherit
    @abstractmethod
    def area(self) -> float: ...

class Notifier(Protocol):          # structural: any class
    def send(self, msg: str) -> None: ...   # with .send() fits
```

super() calls the **next class in the MRO**, not simply "my parent". That's what makes mixins work.

ABC: subclasses that forget an abstract method can't even be created.

Protocol: duck typing with type checking. Great for classes you don't own.

LESSON · COMPOSITION & SOLID

Build from parts, inject the details

```
class Checkout:
    def __init__(self, discount: Discount, notifier: Notifier):
        self.discount = discount        # HAS-A, injected
        self.notifier = notifier

    def pay(self, items):
        total = self.discount.apply(sum(p for _, p in items))
        self.notifier.send(f"charged {total:.2f}")

Checkout(PercentOff(20), ConsoleNotifier()).pay(cart)
Checkout(NoDiscount(), FakeNotifier()).pay(cart)  # in tests
```

Single responsibility · **O**pen for extension · **L**iskov · **I**nterface segregation · **D**ependency inversion

IS-A → inheritance. **HAS-A → composition.**

New discount rule? A **new class**. Checkout never changes.

EXERCISE 12 · YOUR TURN

A Vector class that feels built-in

The task

1	`repr`, `+`, `v * 2` and `2 * v`
2	`abs(v)`, a read-only **length** property
3	`==` and **hashable** (works in sets)
4	`x, y = v`, `bool()`, `Vector.from_tuple()`

```
v = Vector(3, 4)
v + Vector(1, 1)   # Vector(4, 5)
2 * v              # Vector(6, 8)
abs(v)             # 5.0
x, y = v
```

Starter: exercises/ex12_vector.py
Solution: solutions/ex12_vector.py

> Done? Open a **Pull Request** and I'll review your code.

MODULE

13

Errors, context managers & logging

Fail safely, clean up always, leave a clear trail.

IN THIS MODULE

Exceptions like a pro	01_exception_hierarchy.py
Context managers	02_context_managers.py
logging	03_logging.py
Demo: robust order importer	04_demo_order_importer.py
Exercise: safe bank transfers	ex13_bank_transfer.py

Your own exception family

```
class ShopError(Exception): ...          # the family
class OutOfStock(ShopError):
    def __init__(self, item, left):
        super().__init__(f"only {left} x {item} left")
        self.item, self.left = item, left   # extra data

try:
    buy("laptop", 5)
except OutOfStock as e:      # most specific FIRST
    print(e.left)
except ShopError as e:       # then the family
    ...

raise ShopError("config broken") from err   # keep the cause
```

Catch as **specific** as possible; a parent catches all its children.

`raise ... from e` keeps the original error in the traceback.

EAFP: try it, handle the exception. The Pythonic default.

⚠️ Never `except: pass`. It also catches Ctrl+C and hides bugs forever.

Setup and clean-up, guaranteed

```
@contextmanager
def transaction(db):
    snapshot = dict(db)
    try:
        yield                   # the with-block runs here
    except Exception:
        db.clear(); db.update(snapshot)   # roll back
        raise                   # never swallow the error

with transaction(accounts):
    accounts["raj"] += 30
    withdraw(accounts, "vishal", 30)
```

`with x:` → `__enter__`, the body, then `__exit__`, which **always** runs.

Code before `yield` = enter, after = exit.

Files, locks, DB transactions, timers, temp folders, `suppress(FileNotFoundError)`

LESSON · LOGGING

print() for grown-ups

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)-8s %(name)s: %(message)s")
log = logging.getLogger(__name__)    # one per module

log.debug("hidden below INFO")
log.info("order %s created", 1042)   # %s, not f-strings
log.warning("stock low: %d left", 2)
try:
    1 / 0
except ZeroDivisionError:
    log.exception("discount failed")  # + traceback
```

> Levels: **DEBUG < INFO < WARNING < ERROR < CRITICAL**.

> Configure **once** at start-up; add handlers for files or the cloud.

> `%s` arguments: the text is only built if the message is shown.

EXERCISE 13 · YOUR TURN

Safe bank transfers

The task

1. `InsufficientFunds(BankError)` with `.account .balance .amount`
2. `withdraw()` raises it when the balance is too low
3. `atomic(accounts)`: snapshot, roll back, **re-raise**
4. `transfer()`: no money may appear or disappear

```
transfer(accounts, "raj", "vishal", 500)
# InsufficientFunds: raj has 80
accounts   # unchanged: rolled back
```

Starter: exercises/ex13_bank_transfer.py
Solution: solutions/ex13_bank_transfer.py

> Done? Open a **Pull Request** and I'll review your code.

MODULE

14

Project structure & packaging

From scripts to an installable project with its own command.

IN THIS MODULE

Modules & imports	`01_modules_and_imports.py`
CLIs with argparse	`02_cli_with_argparse.py`
A real package: tasky	`tasky_project/`
Exercise: wordtool CLI	`ex14_wordtool_cli.py`

What import really does

```
import json
# 1. already in sys.modules? reuse it (runs only ONCE)
# 2. otherwise search the folders in sys.path
# 3. run the file, store it in sys.modules

if __name__ == "__main__":   # only when run directly,
    main()                   # not when imported (tests!)

from tasky.store import TaskStore   # absolute: preferred
from .store import TaskStore        # relative: inside a package
```

module = one .py file · **package** = a folder of modules with `__init__.py`

`__name__` is `"__main__"` only for the file you ran.

Avoid `from x import *`: you can't tell where names come from.

LESSON · A REAL PACKAGE

src layout + pyproject.toml

```
tasky_project/
├── pyproject.toml      # name, version, deps, commands
├── src/tasky/
│   ├── __init__.py     # public API, __version__
│   ├── __main__.py     # python -m tasky
│   ├── cli.py          # argparse -> calls store
│   └── store.py        # the logic (no printing)
└── tests/test_store.py

[project.scripts]
tasky = "tasky.cli:main"     # creates the `tasky` command
```

`pip install -e .`: an **editable** install; code changes apply instantly.

Keep **logic** (store.py) apart from the **interface** (cli.py): reusable and testable.

Return a **non-zero exit code** on failure so scripts and CI can tell.

$ tasky add "Record Module 14" · $ tasky done 1 · $ python -m tasky list

EXERCISE 14 · YOUR TURN

Build a CLI with argparse

The task

1	Subcommand `stats TEXT` : words, unique, chars
2	Subcommand `top TEXT --n 3` : the most common words
3	`main(argv=None)` : testable without a terminal
4	Case-insensitive counting; return exit code 0

```
$ wordtool stats "the cat and the hat"
words: 5, unique: 4, chars: 19

$ wordtool top "the cat and the hat" --n 1
the: 2
```

Starter: exercises/ex14_wordtool_cli.py
Solution: solutions/ex14_wordtool_cli.py

> Done? Open a **Pull Request** and I'll review your code.

MODULE

15

Type hints & code quality

Catch bugs before you run anything, and keep code easy to read.

IN THIS MODULE

Type hints	`01_type_hints_basics.py`
Advanced typing	`02_advanced_typing.py`
mypy catches bugs	`03_mypy_catches_bugs.py`
ruff & style	`04_ruff_and_style.py`
Exercise: type it, fix it	`ex15_add_type_hints.py`

LESSON · TYPE HINTS

Say what your code expects

TYPE	MEANS
`list[int]`, `dict[str, float]`	what's **inside** a collection
`float | None`	a float, **or None** (handle it!)
`Literal["info", "error"]`	only these exact values
`TypedDict`	a dict with known keys (JSON)
`Callable[[float], float]`	a function float → float
`TypeVar T`: `list[T] -> T`	generic: str in, str out
`Final`	a constant

```
def find_price(item: str) -> float | None:
    return prices.get(item)
```

> Python does **not** check hints at runtime.
> Your editor and **mypy** do.

LESSON · MYPY

3 hidden crashes, found in one second

```python
def plan_of(user_id: int) -> str:
    user = find_user(user_id)        # dict | None
    return user.get("plan", "free")  # crashes for unknown ids

def label(count: int) -> str:
    if count > 100:
        return "Orders: " + count    # str + int

def bill(total: float) -> None:
    send_invoice(total)              # expects an email str
```

$ mypy 03_mypy_catches_bugs.py

```
Item "None" ... has no attribute "get"
```

```
Unsupported operand types for + ("str" and "int")
```

```
Argument 1 ... incompatible type "float"; expected "str"
```

The code runs fine with today's test data. mypy finds the crashes **before** production does.

LESSON · RUFF

Lint for bugs, format for consistency

```
import os, sys                          # E401 + F401 unused
def messy_total( items ,discount = None ):
    for i in range(len(items)):
        if items[i]["price"] == None:   # E711
            continue
    try: ...
    except:                             # E722 bare except
        return 0

$ ruff check file.py      # list problems
$ ruff check --fix        # fix the safe ones
$ ruff format             # rewrite the layout
```

> **Linter**: likely bugs and bad patterns. **Formatter**: layout only.

> Configure once in `pyproject.toml` or `ruff.toml`.

> Tools can't name things for you: `unpaid_invoices`, not `lst2`.

EXERCISE 15 · YOUR TURN

Type it, let mypy find the bug

The task

1. Add hints to every function, with a **TypedDict** for orders
2. Run `mypy --strict`
3. Fix the real bug mypy reports
4. Checks AND mypy --strict must pass

```
$ mypy --strict ex15_add_type_hints.py
error: Value of type "Order | None"
       is not indexable
# report([]) would crash!
```

Starter:
exercises/ex15_add_type_hints.py
Solution:
solutions/ex15_add_type_hints.py

> Done? Open a **Pull Request** and I'll review your code.

MODULE

16

Testing with pytest

Prove your code works, and keep it working when it changes.

IN THIS MODULE

pytest basics	`test_01_basics.py`
Fixtures & parametrize	`test_02_fixtures_and_parametrize.py`
Fakes, mocks & monkeypatch	`test_03_mocking.py`
TDD	`test_04_tdd.py`
Exercise: tests that catch bugs	`ex16_test_password.py`

LESSON · PYTEST BASICS

Plain assert, nothing else to learn

```python
def test_discount_takes_percentage_off():
    price = 100.0                          # arrange
    result = apply_discount(price, 20)     # act
    assert result == 80.0                  # assert

def test_over_100_is_rejected():
    with pytest.raises(ValueError, match="between 0 and 100"):
        apply_discount(10, 150)

def test_floats():
    assert 0.1 + 0.2 == pytest.approx(0.3)
```

> pytest runs files `test_*.py` and functions `test_*`.

> A failed assert shows **both values**: `assert 6.0 == 7`.

> Test the **edges**: 0, empty, the limit, invalid input.

$ pytest 16_testing -v → 31 passed

LESSON · FIXTURES & PARAMETRIZE

Reusable setup, many cases

```python
@pytest.fixture                          # in conftest.py: shared
def sample_cart():
    return {"book": (12.5, 2), "pen": (1.2, 3)}

def test_total(sample_cart):             # asked for by NAME
    assert cart_total(sample_cart) == pytest.approx(28.6)

@pytest.mark.parametrize(("price", "pct", "expected"), [
    (100, 10, 90.0),
    (19.99, 50, 9.99),                  # floats!
])
def test_cases(price, pct, expected):
    assert apply_discount(price, pct) == expected
```

A fixture is built **fresh for every test**, so tests can't affect each other.

`tmp_path` : a built-in fixture, a temp folder per test.

Each parameter set is reported as its **own** test.

LESSON · MOCKS & TDD

No network in unit tests · red → green → refactor

```python
def describe_weather(city, fetch=fetch_json): ...

def test_hot_city():
    fake = Mock(return_value={"temp_c": 31.4})
    assert describe_weather("Mumbai", fetch=fake) == "Mumbai: 31°C, hot"
    fake.assert_called_once()

def test_api_down():
    fake = Mock(side_effect=TimeoutError("down"))
    with pytest.raises(TimeoutError):
        describe_weather("Oslo", fetch=fake)
```

> Real APIs are slow, need a network, and change: tests become **flaky**.

> **Inject** the dependency, or `monkeypatch` it for one test.

> **TDD**: write a failing test → make it pass → refactor. `slugify()` was built that way.

EXERCISE 16 · YOUR TURN

Write tests that catch bugs

The task

1	`is_strong_password()` is correct: test every rule
2	8+ chars · upper · lower · digit · no spaces · TypeError
3	Each test breaks **only one** rule
4	Your tests must catch **6/6** planted bugs

```
$ python ex16_test_password.py
Your tests caught 6/6 bugs.
All checks passed!
```

Starter:
exercises/ex16_test_password.py
Solution:
solutions/ex16_test_password.py

> Done? Open a **Pull Request** and I'll review your code.

MODULE

17

Concurrency

Threads for waiting, processes for calculating, asyncio for thousands of tasks.

IN THIS MODULE

Threads	`01_threads.py`
Processes	`02_processes.py`
asyncio	`03_asyncio_basics.py`
Demo: price checker	`04_demo_price_checker.py`
Exercise: parallel downloads	`ex17_parallel_downloads.py`

LESSON · CHOOSING

Waiting or calculating?

WORK	EXAMPLE	USE	MEASURED
I/O-bound (waiting)	downloads, DB, files	**threads** or **asyncio**	4.0 s → **0.5 s**
CPU-bound (calculating)	maths, images, parsing	**processes**	5.1 s → **2.1 s**
CPU-bound with threads		❌ the GIL	5.1 s → **6.1 s**

```
with ThreadPoolExecutor(8) as pool:
    results = list(pool.map(download, files))

if __name__ == "__main__":     # required!
    with ProcessPoolExecutor() as pool:
        totals = list(pool.map(count_primes, jobs))
```

> The **GIL** lets one thread run Python at a time per process. Separate processes each have their own.

Shared data needs a lock

```
counter = 0
lock = threading.Lock()

def safe_add(n):
    global counter
    for _ in range(n):
        with lock:              # one thread at a time
            counter += 1

# 4 threads x 1000:
#   without the lock: 1204   <- updates lost!
#   with the lock:    4000
```

A **race condition**: two threads read, both add 1, both write. One update is lost.

`with lock:` makes the read-modify-write happen as one step.

Simplest of all: **don't share**. Return results from the pool instead.

LESSON · ASYNCIO

Thousands of waiting tasks on one thread

```python
async def fetch(name, s):
    await asyncio.sleep(s)        # pause, let others run
    return name

async def main():
    results = await asyncio.gather(fetch("x", 1), fetch("y", 1))   # ~1 s
    await asyncio.wait_for(fetch("slow", 5), timeout=0.5)         # TimeoutError
    limit = asyncio.Semaphore(10)                                 # max 10 at once

asyncio.run(main())
```

> Switches happen **only at await**, so race conditions are rarer.

> `gather` = run concurrently · `wait_for` = timeout · `Semaphore` = limit.

⚠️ One blocking call (`time.sleep`, `requests.get`) freezes the whole loop.

EXERCISE 17 · YOUR TURN

Parallel downloads

The task

1	10 downloads × 0.3 s in **< 1 s** with a **ThreadPoolExecutor**
2	The same with **asyncio.gather**
3	Count words with a **ProcessPoolExecutor**
4	Keep the results in input order

```
$ python ex17_parallel_downloads.py
threads: 0.31 s, asyncio: 0.31 s
All checks passed!
```

Starter:
exercises/ex17_parallel_downloads.py
Solution:
solutions/ex17_parallel_downloads.py

> Done? Open a **Pull Request** and I'll review your code.

MODULE

18

Data, APIs & databases

Files, web APIs, SQL, and your own REST API with FastAPI.

IN THIS MODULE

Files, CSV & JSON	`01_files_csv_json.py`
Calling web APIs	`02_http_requests.py`
SQLite	`03_sqlite.py`
Your own REST API	`04_fastapi_app.py`
Exercise: a notes database	`ex18_notes_repository.py`

LESSON · WEB APIS

Call an API safely

```
try:
    resp = requests.get(
        "https://api.github.com/search/repositories",
        params={"q": "language:python", "per_page": 3},
        timeout=10)                    # ALWAYS
    resp.raise_for_status()            # 4xx/5xx -> error
    data = resp.json()
except requests.Timeout:
    ...
except requests.RequestException:   # offline, DNS, HTTP
    ...
```

> `params=` encodes query values correctly; never glue strings.

> Tokens live in **environment variables**, never in the code.

> `pathlib` + `csv.DictReader` + `json` for files: lesson 01.

LESSON · SQLITE

SQL with ? parameters and transactions

```
conn.execute("SELECT * FROM products WHERE name = ?", (name,))

user_input = "x' OR '1'='1"
f"... WHERE name = '{user_input}'"   # matches 3 rows: INJECTION
"... WHERE name = ?", (user_input,)  # matches 0 rows: safe

with conn:                                # all or nothing
    conn.execute("UPDATE products SET stock = stock - 1 ...")
    conn.execute("INSERT ... duplicate name")   # IntegrityError
# -> the UPDATE was rolled back too
```

> SQLite: a full SQL database in **one file**, built into Python.

> `conn.row_factory = sqlite3.Row` → `row["name"]`.

> The same ideas work for PostgreSQL and MySQL.

LESSON · FASTAPI

Type hints become a validated web API

```
class TodoIn(BaseModel):
    title: str = Field(min_length=1, max_length=100)

@app.post("/todos", status_code=201)
def create_todo(todo: TodoIn) -> Todo: ...

@app.get("/todos/{todo_id}")
def read_todo(todo_id: int) -> Todo:   # 404 if missing
    ...

# POST {"title": ""}  -> 422 (validated for you)
# $ uvicorn 04_fastapi_app:app --reload  -> /docs
```

pydantic models validate the JSON body automatically.

Status codes: **201** created · **204** no content · **404** not found · **422** invalid.

`TestClient` calls the API in tests without a server.

EXERCISE 18 · YOUR TURN

A notes database

The task

1	`NotesRepository(path)` creates the table
2	`add / get / search / delete / count`
3	Tags stored as **JSON** text
4	Only **?** parameters: the injection check must pass

```
repo = NotesRepository(":memory:")
nid = repo.add("Shopping", "milk", tags=["home"])
repo.search("MILK")    # case-insensitive
repo.search("x' OR '1'='1")   # []
```

Starter:
exercises/ex18_notes_repository.py
Solution:
solutions/ex18_notes_repository.py

> Done? Open a **Pull Request** and I'll review your code.

MODULE

19

Performance & internals

Understand what Python does under the hood, then measure before you optimise.

IN THIS MODULE

The memory model	`01_memory_model.py`
Measure first	`02_measure_first.py`
Speed tricks that matter	`03_speed_tricks.py`
Memory tricks	`04_memory_tricks.py`
Exercise: make it fast	`ex19_make_it_fast.py`

Names point to objects

```
a = [1, 2, 3]
b = a               # 2 names, 1 object
b.append(4)         # a sees it too

x = 10; y = x
y += 1              # ints are immutable: y -> NEW object

def add(lst): lst.append(1)   # the object is passed,
add(a)                         # so the caller's list changes

int("1000") is int("1000")     # False: use == for values
```

> Every object has an **identity** (id), a **type** and a **value**.

> **Reference counting**: freed the moment the count hits 0; `gc` cleans up cycles.

> Use `is` only for `None`, `True`, `False`.

LESSON · SPEED

Measure first, then fix the biggest thing

FIX	MEASURED SPEED-UP
Better data structure: list `in` → set/dict	**309×**
Cache a slow pure function: `@lru_cache`	**91×**
`"".join()` instead of `+=` in a loop	**22×**
Built-ins (`sum`, `sorted`, `max`) instead of loops	**4×**
Compute once, outside the loop	**3×**
`__slots__` on millions of objects	24 → 17.6 MB

```
$ python -m cProfile -s cumtime app.py

ncalls  cumtime  function
     1    0.222  report
     1    0.172  slow_unique_customers  <- here!
```

> **timeit** compares snippets · **cProfile** finds slow functions · **tracemalloc** finds memory-hungry lines.

EXERCISE 19 · YOUR TURN

Make it fast

The task

1	`find_duplicates` : O(n²) → O(n) with **Counter**
2	`common_customers` : use **set** intersection
3	`fib(300)` : stop recomputing with **lru_cache**
4	Each must fit a **0.1 s** budget

```
# before
find_duplicates(big)   # 1.28 s
# after
find_duplicates(big)   # 0.004 s
```

Starter: exercises/ex19_make_it_fast.py
Solution: solutions/ex19_make_it_fast.py

> Done? Open a **Pull Request** and I'll review your code.

MODULE

20

Capstone project

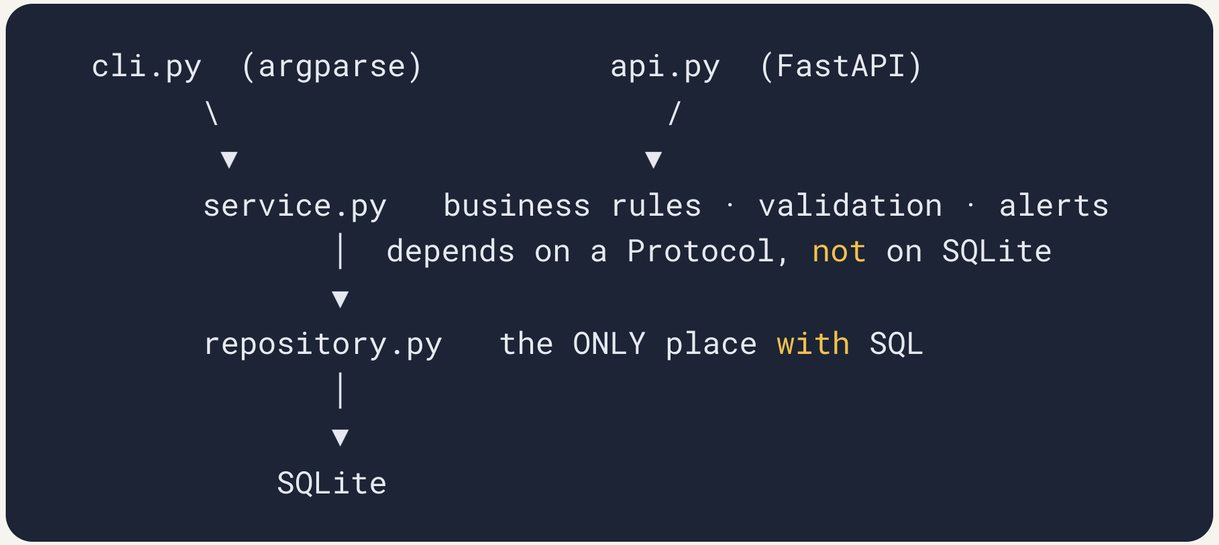
budget: a tested, typed, packaged expense tracker with a CLI and a REST API.

IN THIS MODULE

The project	`budget_project/`
Architecture	`budget_project/README.md`
Tests	`budget_project/tests/`
Challenges	`ex20_capstone_challenges.md`
Final exam	`20_capstone/QUIZ.md`

CAPSTONE · ARCHITECTURE

Clean layers: each talks only to the one below



> The **service** doesn't know whether it's called from a terminal or over HTTP.

> Tests inject an **in-memory** repository: fast and isolated.

> `pytest` · `mypy --strict` · `ruff`: all green.

CAPSTONE · WHERE EVERY MODULE SHOWS UP

Everything you learned, in one project

MODULE	IN BUDGET
08 Pythonic · 09 Data structures	comprehensions, `sorted(key=)`, frozen dataclass, defaultdict
10 Functions · 12 OOP	FastAPI dependencies, Protocol + dependency injection
13 Errors & logging	`BudgetError`, `raise ... from`, `getLogger(__name__)`
14 Packaging	src layout, `pyproject.toml`, the `budget` command
15 Typing · 16 Testing	mypy --strict, ruff, 9 tests with fixtures + TestClient
18 Data & APIs	SQLite repository, FastAPI with pydantic

```
$ budget add 320 food "groceries"
$ budget summary --month 2026-11
2026-11: total 1315.50
  rent    900.00
  food    415.50  <- over budget!
```


github.com/CodeMitVishal/python-pro

EXERCISE 20 · YOUR TURN

Capstone challenges

The task

1	⭐ `budget list` and a transport limit
2	⭐⭐ CSV export · filtered `GET /expenses` · limits from JSON
3	⭐⭐⭐ recurring expenses (generators) · currency + lru_cache
4	⭐⭐⭐ a second repository behind the same Protocol

```
# for EVERY challenge:
# 1. write the test first
# 2. make it pass
# 3. keep pytest, mypy, ruff green
```

Starter:
exercises/ex20_capstone_challenges.md

> Done? Open a **Pull Request** and I'll review your code.

YOU MADE IT

From Foundations to Pro 🎓

Take the **final exam** in 20_capstone/QUIZ.md, build the capstone challenges, and share your project on LinkedIn.

📺 youtube.com/@CodeMitVishal

⭐ github.com/CodeMitVishal/python-pro

☕ buymeacoffee.com/codemitvishal